ANALYTICAL PROBLEM SOLVING

- The warehouse is represented as a 5×5 grid.
- The robot starts at cell (1,1) and must deliver a package to cell (5,5).
- Blocked cells: (2,2), (2,3), (3,3), (4,2), and (4,4).
- The robot can move Up, Down, Left, or Right; diagonal movement is not allowed.
- Every movement has a uniform cost of 1.

a. PEAS Description

PEAS stands for Performance measure, Environment, Actuators, and Sensors.

Component	Description
Performance Measure	Deliver the package to (5,5), minimize the number of movements, avoid blocked cells, avoid unnecessary movements, and reach the destination safely.
Environment	A 5×5 warehouse grid containing the robot, destination, and blocked cells representing shelving units.
Actuators	Movement controls that allow the robot to move Up, Down, Left, or Right.
Sensors	Sensors to detect the robot's current position, nearby cells, obstacles/shelves, and destination.

PEAS in simple form

- P – Performance: Reach (5,5) with minimum movement cost.
- E – Environment: 5×5 warehouse grid with obstacles.
- A – Actuators: Up, Down, Left, Right movement.
- S – Sensors: Detect current location, obstacles, free cells, and goal.

b. Type of Intelligent Agent

Best choice: Goal-Based Agent

A goal-based agent is most suitable for this warehouse robot because it has a clearly defined goal: reach cell (5,5) and deliver the package. The robot must evaluate possible movements and choose a sequence of actions that leads from the initial position to the goal while avoiding blocked cells.

Why not the other types?

- Simple Reflex Agent: Reacts only to the current situation and does not maintain enough information to systematically find a path.
- Model-Based Agent: Can maintain an internal model of the environment, but the main requirement here is goal-directed path finding.
- Goal-Based Agent: Best suited because the robot has a specific destination and must plan actions to reach it.
- Utility-Based Agent: Could be used if factors such as battery consumption, time, safety, and distance had different priorities. Here, the main objective is reaching the goal with minimum movement cost.

Therefore, the robot is best modeled as a Goal-Based Agent using a search algorithm such as BFS.

c. Formal Search Problem

1. Initial State

Initial State = (1,1)

2. Goal State

Goal State = (5,5)

3. Actions

From any valid cell, the robot can perform Up, Down, Left, or Right. An action is allowed only if the resulting cell is inside the 5×5 grid and is not blocked.

4. Transition Model

The transition model gives the new position after an action. For example, (1,1) + Down → (2,1), and (1,1) + Right → (1,2). A move into a blocked cell, such as (2,1) + Right → (2,2), is not allowed.

d. Breadth-First Search (BFS)

BFS explores nodes level by level. Since every movement has the same cost of 1, BFS finds a shortest path.

Neighbor order used: Down → Right → Up → Left.

Initial Queue: [(1,1)]

PYTHON PROGRAM:

BFS.py - C:\Users\ggnan\OneDrive\Desktop\HARSH AI\BFS.py (3.12.10)

File Edit Format Run Options Window Help

```
from collections import deque
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))
start = tuple(map(int, input("Enter start position (row col): ").split()))
goal = tuple(map(int, input("Enter goal position (row col): ").split()))
n = int(input("Enter number of blocked cells: "))
blocked = set()
for i in range(n):
    cell = tuple(map(int, input(f"Enter blocked cell {i + 1} (row col): ").split()))
    blocked.add(cell)
directions = [
    (1, 0),
    (0, 1),
    (-1, 0),
    (0, -1)
]
queue = deque([(start, [start])])
visited = {start}
print("\nBFS Node Expansion:")
while queue:
    current, path = queue.popleft()
    print("Expanded:", current)
    print("Queue:", [item[0] for item in queue])
    if current == goal:
        print("\nGoal Reached!")
        print("BFS Path:")
        print(" -> ".join(map(str, path)))
        print("Total Cost:", len(path) - 1)
        break
    for dr, dc in directions:
        next_row = current[0] + dr
        next_col = current[1] + dc
        next_cell = (next_row, next_col)
        if (1 <= next_row <= rows and
            1 <= next_col <= cols and
            next_cell not in blocked and
            next_cell not in visited):
            visited.add(next_cell)
            queue.append((next_cell, path + [next_cell]))
else:
    print("No path found.")
```

OUTPUT:

```
========== RESTART: C:\Users\ggnan\OneDrive\Desktop\HARSH AI\BFS.py ==========
Enter number of rows: 5
Enter number of columns: 5
Enter start position (row col): 1 1
Enter goal position (row col): 5 5
Enter number of blocked cells: 5
Enter blocked cell 1 (row col): 2 2
Enter blocked cell 2 (row col): 2 3
Enter blocked cell 3 (row col): 3 3
Enter blocked cell 4 (row col): 4 2
Enter blocked cell 5 (row col): 4 4
```

```
BFS Node Expansion:
Expanded: (1, 1)
Queue: []
Expanded: (2, 1)
Queue: [(1, 2)]
Expanded: (1, 2)
Queue: [(3, 1)]
Expanded: (3, 1)
Queue: [(1, 3)]
Expanded: (1, 3)
Queue: [(4, 1), (3, 2)]
Expanded: (4, 1)
Queue: [(3, 2), (1, 4)]
Expanded: (3, 2)
Queue: [(1, 4), (5, 1)]
Expanded: (1, 4)
Queue: [(5, 1)]
Expanded: (5, 1)
Queue: [(2, 4), (1, 5)]
Expanded: (2, 4)
Queue: [(1, 5), (5, 2)]
Expanded: (1, 5)
Queue: [(5, 2), (3, 4), (2, 5)]
Expanded: (5, 2)
Queue: [(3, 4), (2, 5)]
Expanded: (3, 4)
Queue: [(2, 5), (5, 3)]
Expanded: (2, 5)
Queue: [(5, 3), (3, 5)]
Expanded: (5, 3)
Queue: [(3, 5)]
Expanded: (3, 5)
Queue: [(5, 4), (4, 3)]
Expanded: (5, 4)
Queue: [(4, 3), (4, 5)]
Expanded: (4, 3)
Queue: [(4, 5), (5, 5)]
Expanded: (4, 5)
Queue: [(5, 5)]
Expanded: (5, 5)
Queue: []

Goal Reached!
BFS Path:
(1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (5, 1) -> (5, 2) -> (5, 3) -> (5, 4) -> (5, 5)
Total Cost: 8
```

e. Depth-First Search (DFS)

DFS explores as far as possible along one branch before backtracking.

Neighbor order used: Down → Right → Up → Left. For the stack, neighbors are pushed in reverse order so Down is explored first.

Initial Stack: [(1,1)]

PYTHON PROGRAM:

DFS.py - C:/Users/ggnan/OneDrive/Desktop/HARSH AI/DFS.py (3.12.10)

File Edit Format Run Options Window Help

```
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))
start = tuple(map(int, input("Enter start position (row col): ").split()))
goal = tuple(map(int, input("Enter goal position (row col): ").split()))
n = int(input("Enter number of blocked cells: "))
blocked = set()
for i in range(n):
    cell = tuple(map(int, input(f"Enter blocked cell {i + 1} (row col): ").split()))
    blocked.add(cell)
directions = [
    (1, 0),
    (0, 1),
    (-1, 0),
    (0, -1)
]
stack = [(start, [start])]
visited = {start}
print("\nDFS Node Expansion:")
while stack:
    current, path = stack.pop()
    print("Expanded:", current)
    print("Stack:", [item[0] for item in stack])
    if current == goal:
        print("\nGoal Reached!")
        print("DFS Path:")
        print(" -> ".join(map(str, path)))
        print("Total Cost:", len(path) - 1)
        break
    for dr, dc in reversed(directions):
        next_row = current[0] + dr
        next_col = current[1] + dc
        next_cell = (next_row, next_col)
        if (1 <= next_row <= rows and
            1 <= next_col <= cols and
            next_cell not in blocked and
            next_cell not in visited):
            visited.add(next_cell)
            stack.append((next_cell, path + [next_cell]))
else:
    print("No path found.")
```

OUTPUT:

```
========== RESTART: C:/Users/ggnan/OneDrive/Desktop/HARSH AI/DFS.py ==========
Enter number of rows: 5
Enter number of columns: 5
Enter start position (row col): 1 1
Enter goal position (row col): 5 5
Enter number of blocked cells: 5
Enter blocked cell 1 (row col): 2 2
Enter blocked cell 2 (row col): 2 3
Enter blocked cell 3 (row col): 3 3
Enter blocked cell 4 (row col): 4 2
Enter blocked cell 5 (row col): 4 4

DFS Node Expansion:
Expanded: (1, 1)
Stack: []
Expanded: (2, 1)
Stack: [(1, 2)]
Expanded: (3, 1)
Stack: [(1, 2)]
Expanded: (4, 1)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 1)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 2)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 3)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 4)
Stack: [(1, 2), (3, 2), (4, 3)]
Expanded: (5, 5)
Stack: [(1, 2), (3, 2), (4, 3)]

Goal Reached!
DFS Path:
(1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (5, 1) -> (5, 2) -> (5, 3) -> (5, 4) -> (5, 5)
Total Cost: 8
```

f. BFS vs DFS Comparison

Property	BFS	DFS
Completeness	Complete if the branching factor is finite.	Complete in finite search spaces with appropriate visited-state handling.
Optimality	Optimal for equal step costs.	Not optimal in general.
Time Complexity	O(b^d)	O(b^m)
Space Complexity	O(b^d)	O(bm)
Search Strategy	Level by level	Depth first
Memory Requirement	Higher	Lower
Path in this problem	Cost = 8	Cost = 8
Suitability	Highly suitable	Possible, but less reliable

Where:

b = branching factor, d = depth of the shallowest goal, and m = maximum depth of the search tree.

Suitable Algorithm:

BFS is more suitable for this warehouse delivery problem. The main reason is that every movement has the same cost of 1. BFS explores paths in increasing order of their length. Therefore, when BFS first reaches the destination, it has found a minimum-cost path. Although DFS also produces a cost-8 path in this particular example, DFS does not generally guarantee the shortest path.

Conclusion:

BFS is preferred because the warehouse robot has uniform movement costs and needs to reach the destination using the shortest possible route. BFS is complete and optimal for this type of problem, whereas DFS may find a solution but does not guarantee the shortest path.